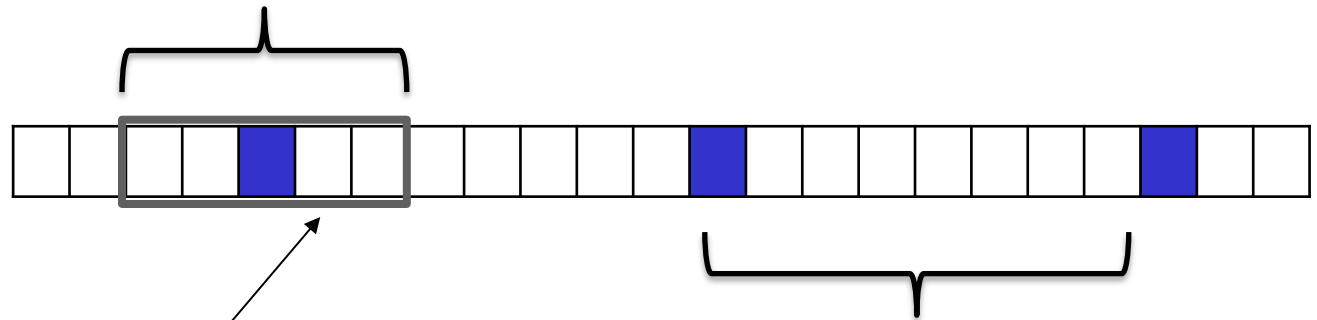


Vektor Zugriff mit Stride k

Cache Line/Block
der Größe $B=5$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*k];  
}
```

Speicher mit
Vektor $a[]$



Wort $a[s]$

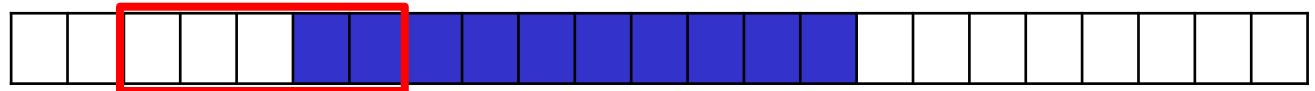
Stride $k=8$

Vektor Zugriff mit Stride $k=1$

Iteration $i=0$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Miss



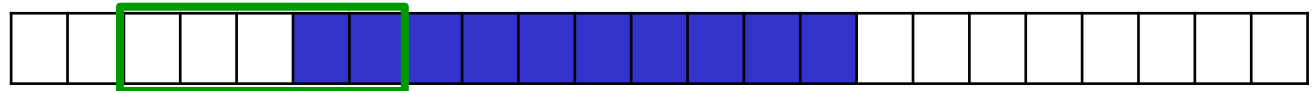
Wort $a[s+0*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=1$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Hit



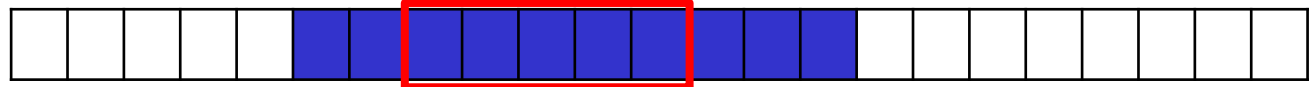
Wort $a[s+1*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=2$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Miss



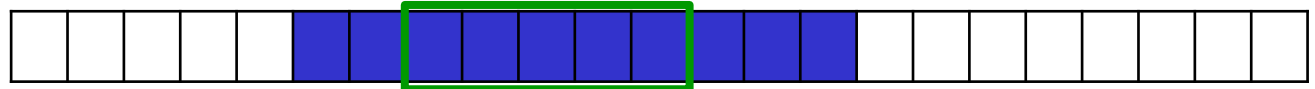
Wort $a[s+2*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=3$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Hit



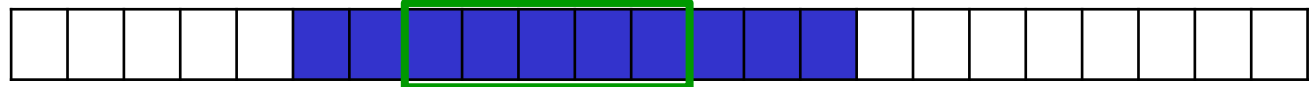
Wort $a[s+3*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=4$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Hit



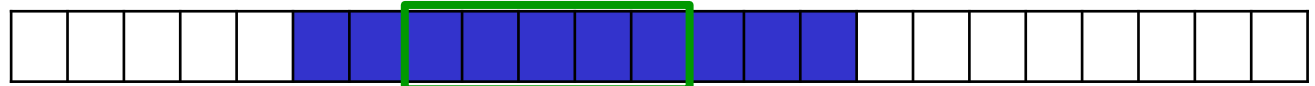
Wort $a[s+4*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=5$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Hit



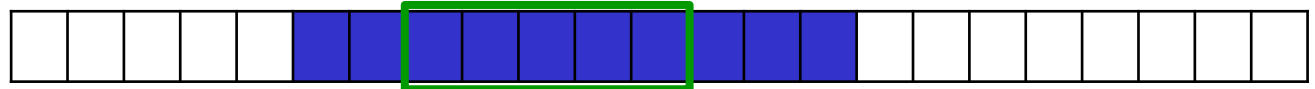
Wort $a[s+5*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=6$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Hit



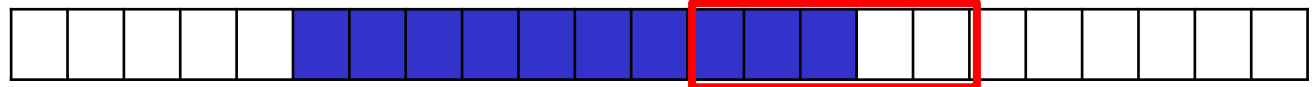
Wort $a[s+6*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=7$

```
for (i=0; i<N; i++){  
    ... = ... a[s + i*0];  
}
```

Cache Miss



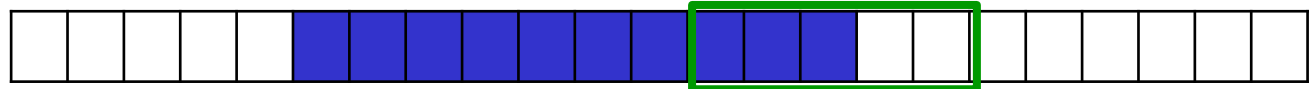
Wort $a[s+7*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=8$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Hit



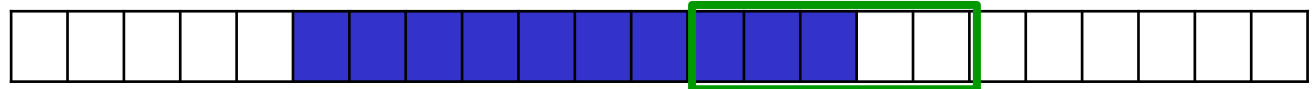
Wort $a[s+8*1]$

Vektor Zugriff mit Stride $k=1$

Iteration $i=9$

```
for (i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Cache Hit



Wort $a[s+9*1]$

Vektor Zugriff mit Stride **k=1**

N=10 Zugriffe auf Vektor a[]

mit **Stride Size k=1** und

Cache-Line Size B=5

bei beliebiger Cache Größe $M > B$

erzeugt **3 Cache Miss** und **7 Cache Hits**.

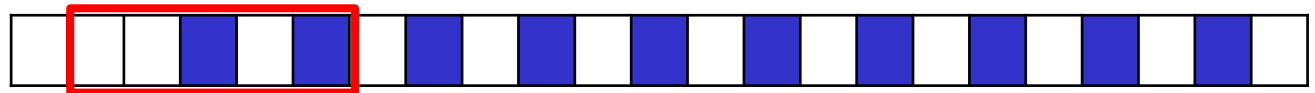
```
for(i=0; i<N; i++){  
    ... = ... a[s + i*1];  
}
```

Vektor Zugriff mit Stride $k=2$

Iteration $i=0$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Miss



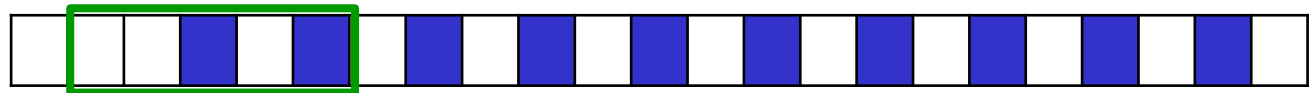
Wort $a[s+0*2]$

Vektor Zugriff mit Stride $k=2$

Iteration $i=1$

```
for (i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Hit



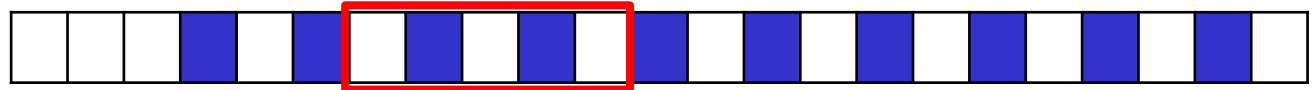
Wort $a[s+1*2]$

Vektor Zugriff mit Stride $k=2$

Iteration $i=2$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Miss



Wort $a[s+2*2]$

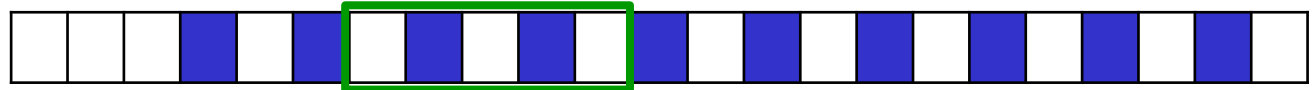
Stride $k=2$

Vektor Zugriff mit Stride $k=2$

Iteration $i=3$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Hit



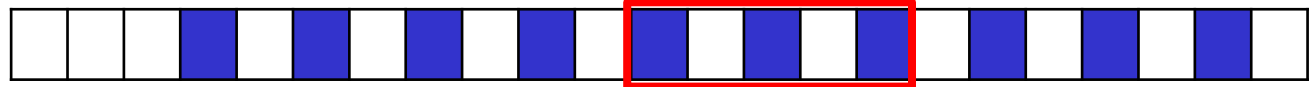
Wort $a[s+3*2]$

Vektor Zugriff mit Stride $k=2$

Iteration $i=4$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Miss



Wort $a[s+4*2]$

Vektor Zugriff mit Stride $k=2$

Iteration $i=5$

```
for (i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Hit



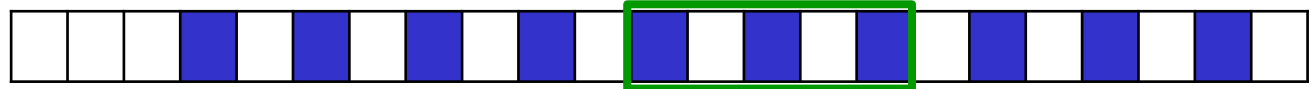
Wort $a[s+5*2]$

Vektor Zugriff mit Stride $k=2$

Iteration $i=6$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Hit



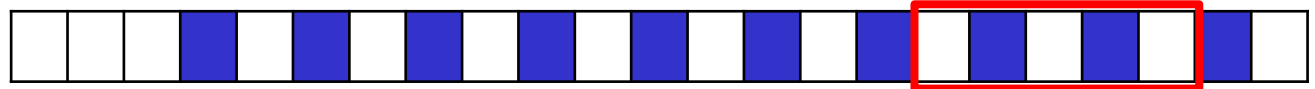
Wort $a[s+6*2]$

Vektor Zugriff mit Stride $k=2$

Iteration $i=7$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Miss



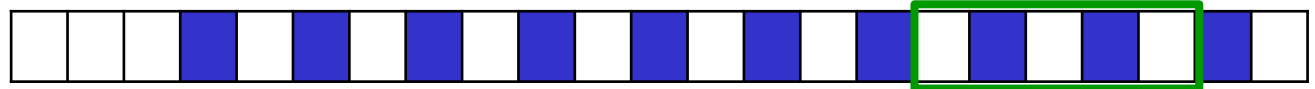
Wort $a[s+7*2]$

Vektor Zugriff mit Stride $k=2$

Iteration $i=8$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Hit



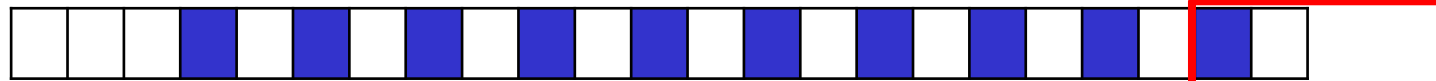
Wort $a[s+8*2]$

Vektor Zugriff mit Stride $k=2$

Iteration $i=9$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Cache Miss



Wort $a[s+9*2]$

Vektor Zugriff mit Stride $k=2$

$N=10$ Zugriffe auf Vektor $a[]$

mit Stride Size $k=2$ und

Cache-Line Size $B=5$

bei beliebiger Cache Größe $M > B$

erzeugt 5 Cache Miss und 5 Cache Hits.

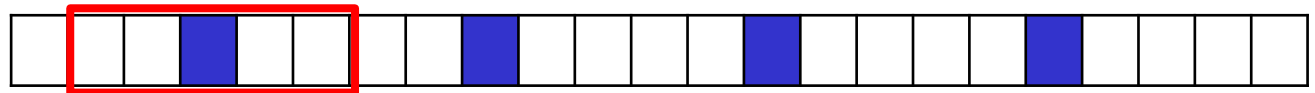
```
for(i=0; i<N; i++){  
    ... = ... a[s + i*2];  
}
```

Vektor Zugriff mit Stride $k=5$

Iteration $i=0$

```
for (i=0; i<N; i++){  
    ... = ... a[s + i*5];  
}
```

Cache Miss



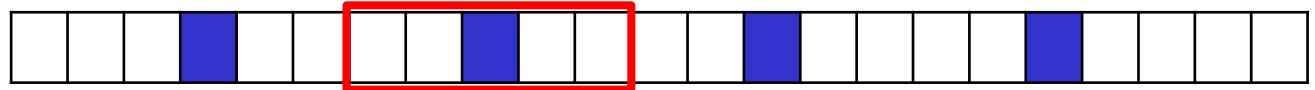
Wort $a[s+0*5]$

Vektor Zugriff mit Stride $k=5$

Iteration $i=1$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*5];  
}
```

Cache Miss



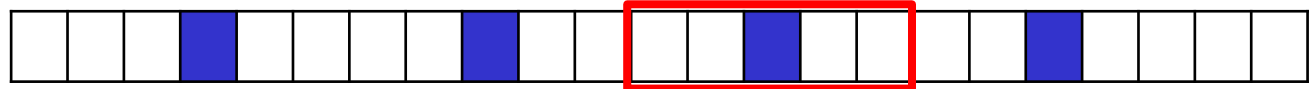
Wort $a[s+1*5]$

Vektor Zugriff mit Stride $k=5$

Iteration $i=3$

```
for (i=0; i<N; i++){  
    ... = ... a[s + i*5];  
}
```

Cache Miss



Wort $a[s+3*5]$

Vektor Zugriff mit Stride $k=5$

$N=10$ Zugriffe auf Vektor $a[]$

mit Stride Size $k=5$ und

Cache-Line Size $B=5$

bei beliebiger Cache Größe $M > B$

erzeugt 10 Cache Miss und 0 Cache Hits.

Ähnlich für $k=6,7,8,\dots$

```
for(i=0; i<N; i++){  
    ... = ... a[s + i*5];  
}
```

k	0	1	2	3	4	5	6	7	...
Miss	1	3	5			10	10	10	10
Hit	9	7	5			0	0	0	0

Vektor Zugriff mit Stride k

N Zugriffe auf Vektor $a[]$ mit
Stride Size k ohne Shift bei ($s=0$)
und Cache-Line Size B bei
beliebiger Cache Größe $M > B$ erzeugt

```
for(i=0; i<N; i++){  
    ... = ... a[0+i*k];  
}
```

$\min\left(N, \left\lfloor \frac{(N-1)k}{B} \right\rfloor + 1\right)$ Cache Miss

und

$N - \min\left(N, \left\lfloor \frac{(N-1)k}{B} \right\rfloor + 1\right)$ Cache Hits.

Hinweis zu Aufgabe 5.1

$k=2, n=7$

Index	0	1	2	3	4	5	6	...
Value	$(0+k)\%n$	$(1+k)\%n$	$(2+k)\%n$	$(3+k)\%n$	$(4+k)\%n$	$(5+k)\%n$	$(6+k)\%n$	...